

Trabajo Práctico Especial

Arcodaci, Tiziano (64637) Trivelloni, Francisco (64141)
tarcodaci@itba.edu.ar ftrivelloni@itba.edu.ar

Pizzuto Beltran, Lorenzo (64123)
lpizzutobeltran@itba.edu.ar

Profesores

Garberoglio, Marcelo Fabio
Kulesz, Sebastian
Stupenengo Faus, Hugo Javier
Axt, Roberto Oscar
Mizrahi, Thomas

Instituto Tecnológico de Buenos Aires (ITBA)
Protocolos de Comunicación

Fecha de entrega: 14/06/2026

Índice

1. Descripción de los protocolos y aplicaciones	3
1.1. Servidor Proxy SOCKSv5	3
1.2. Protocolo de Monitoreo	3
1.2.1. Formato de Request	3
1.2.2. Formato de Response	4
1.2.3. Comandos	4
1.2.4. Códigos de estado	4
1.2.5. Decisiones de diseño	4
1.3. Cliente de Monitoreo	5
2. Documento de diseño	5
2.1. Arquitectura general	5
2.2. Máquina de estados	5
2.3. Módulos	6
2.4. Resolución DNS asíncrona	6
2.5. Half-close y control de flujo	7
2.6. Graceful shutdown	7
3. Pruebas de estrés	7
4. Problemas encontrados	8
5. Limitaciones	9
6. Posibles extensiones	9
7. Conclusiones	10
8. Ejemplos de prueba	10
9. Guía de instalación	14
10. Configuración	15

1. Descripción de los protocolos y aplicaciones

1.1. Servidor Proxy SOCKSv5

Se implementó un servidor proxy que cumple con el protocolo SOCKS versión 5 (RFC 1928) con autenticación por usuario y contraseña (RFC 1929). El servidor maneja múltiples conexiones de forma concurrente usando I/O no bloqueante con `pselect` en un único hilo de ejecución.

Cada conexión pasa por una secuencia de estados: se negocia el método de autenticación, se validan las credenciales, se procesa el pedido de conexión al destino, y finalmente se entra en un relay bidireccional que copia bytes entre el cliente y el servidor de destino hasta que alguno cierre la conexión.

Se soporta únicamente el comando `CONNECT` con direcciones IPv4, IPv6 y nombres de dominio (FQDN). Para la resolución de dominios se utiliza un hilo separado que ejecuta `getaddrinfo` sin bloquear el loop principal. Cuando un dominio resuelve a múltiples direcciones IP y la primera no está disponible, se intenta con las siguientes en orden.

1.2. Protocolo de Monitoreo

Se diseñó un protocolo binario propio que opera sobre TCP en un puerto separado (8080 por defecto). Permite administrar el servidor en tiempo de ejecución: gestionar usuarios, consultar métricas y revisar el registro de accesos.

1.2.1. Formato de Request

VER	CMD	NARGS	LEN_1	LEN_2	PAYLOAD
1B	1B	1B	1B	1B	variable

- VER: versión del protocolo, siempre 0x01.
- CMD: código de la operación solicitada.
- NARGS: cantidad de argumentos (0 a 2).
- LEN_N: largo de cada argumento en bytes.
- PAYLOAD: argumentos concatenados.

1.2.2. Formato de Response

```
+-----+-----+-----+-----+
| VER | STATUS | LEN | PAYLOAD |
+-----+-----+-----+-----+
1B      1B      2B  variable
                     (BE)
```

- STATUS: resultado de la operación.
- LEN: largo del payload en big-endian.

1.2.3. Comandos

CMD	Nombre	Args	Descripción
0x00	AUTH	2	Autenticación (usuario, contraseña)
0x01	ADD_USER	2	Agregar usuario
0x02	DEL_USER	1	Eliminar usuario
0x03	LIST_USERS	0	Listar usuarios registrados
0x04	GET_METRICS	0	Consultar métricas
0x05	GET_ACCESS_LOG	0	Consultar registro de accesos

1.2.4. Códigos de estado

0x00	Éxito
0x01	Autenticación fallida
0x02	Comando desconocido
0x03	Usuario ya existe
0x04	Usuario no encontrado
0x05	Error interno

1.2.5. Decisiones de diseño

Se optó por un formato binario con campos length-prefixed. Este enfoque simplifica el parsing dado que no se requieren delimitadores ni escapeo de caracteres especiales (relevante para contraseñas que pueden contener cualquier byte), y resulta más compacto para la transmisión de métricas numéricas. Además, el formato sigue el mismo patrón que utiliza SOCKS en sus mensajes de negociación (versión + largo + datos), lo que permitió reutilizar el motor de parsers existente.

Se eligió TCP como transporte ya que las operaciones del protocolo son mutadoras (agregar y eliminar usuarios) y se requiere confirmación de que el pedido fue procesado correctamente.

1.3. Cliente de Monitoreo

Se desarrolló una aplicación de línea de comandos (`bin/client`) que se conecta al servidor mediante el protocolo de monitoreo y permite ejecutar operaciones de administración:

```
./bin/client -u admin:pass -m          # métricas
./bin/client -u admin:pass -U          # listar usuarios
./bin/client -u admin:pass -a user:pass # agregar
./bin/client -u admin:pass -d user      # eliminar
```

El cliente resuelve la dirección del servidor vía DNS y soporta failover a múltiples direcciones. Se configura un timeout global de 10 segundos para la operación completa.

Además del modo de un solo comando, el cliente cuenta con un modo interactivo que presenta un menú con vistas dedicadas para métricas, gestión de usuarios y log de accesos. La interfaz se construyó con ncurses, aprovechando colores y posicionamiento de cursor para facilitar la lectura de la información.

El código del cliente se organiza en: `client_args.c` para el parsing de argumentos, `mon_client.c` para la lógica de conexión y comunicación con el servidor, y el directorio `ui/` con vistas separadas para métricas, usuarios y la sesión interactiva.

2. Documento de diseño

2.1. Arquitectura general

El servidor consiste en un único proceso con un event loop basado en `pselect`. Tanto las conexiones SOCKS como las de monitoreo se atienden en el mismo hilo. El único uso de hilos adicionales es para la resolución de nombres de dominio, dado que `getaddrinfo` es una función bloqueante sin alternativa asíncrona portable.

El programa abre dos sockets pasivos TCP:

- Puerto 1080 (configurable): conexiones SOCKS.
- Puerto 8080 (configurable, solo localhost): conexiones de administración.

2.2. Máquina de estados

Cada conexión SOCKS se modela como una máquina de estados finita (usando el motor `stm.c` provisto por la cátedra) que va transitando por las fases del protocolo.

En cada estado, la conexión tiene registrado un interés en el selector (leer, escribir, o ninguno) según lo que necesite hacer a continuación.

En el estado de relay, los intereses se ajustan dinámicamente según el estado de los buffers internos: si el buffer hacia un destino está lleno, se deja de leer del origen correspondiente. Esto evita que un extremo rápido sature la memoria cuando el otro es más lento.

2.3. Módulos

La lógica del servidor se encuentra separada en los siguientes módulos:

- `socks5nio.c`: manejo de sockets, accept, pool de conexiones.
- `socks5.c`: lógica del protocolo SOCKS, STM, relay client-side, hello/auth/request handlers.
- `connect.c`: conexión al servidor de destino (socket, connect no bloqueante, fallback de IPs, origin fd callbacks).
- `dns_resolver.c`: resolución DNS asíncrona en hilo separado.
- `hello.c`, `auth.c`, `request.c`: parsers byte a byte para cada fase del protocolo.
- `mon_nio.c`: server-side del protocolo de monitoreo.
- `mon_parser.c`: parser de requests de monitoreo.
- `user_table.c`: tabla de credenciales con operaciones CRUD.
- `metrics.c`: contadores de conexiones y bytes transferidos.
- `access_log.c`: buffer circular de registros de acceso.

2.4. Resolución DNS asíncrona

Para no bloquear el event loop al recibir un FQDN, se crea un hilo con `pthread_create` que ejecuta `getaddrinfo` de forma bloqueante. Al completarse la resolución, el hilo notifica al selector mediante `selector_notify_block`, que despierta al hilo principal del `pselect` vía señal.

Dado que la estructura de conexión es compartida entre el hilo DNS y el hilo principal, se utiliza un contador de referencias atómico para asegurar que la memoria se libere recién cuando ninguno de los dos la necesite.

2.5. Half-close y control de flujo

Cuando un extremo de la conexión cierra su lado de escritura (se detecta al leer 0 bytes), se vacían los datos pendientes en el buffer y se propaga un `shutdown(SHUT_WR)` al otro extremo. La conexión se cierra completamente recién cuando ambos lados cerraron y los buffers están vacíos.

Después de cada `read` en el relay, se intenta inmediatamente un `write` al fd opuesto mediante la función `relay_flush`. Si el write tiene éxito (caso común cuando el peer está listo), los datos se entregan sin volver al selector — reduciendo el ciclo de `select→read→select→write` a `select→read→write`. Si retorna `EAGAIN`, se registra `OP_WRITE` y se continúa normalmente en la siguiente iteración. Esto reduce la latencia del relay a la mitad en el caso típico sin modificar la semántica del control de flujo.

2.6. Graceful shutdown

Se instalan handlers para `SIGTERM` y `SIGINT` que setean un flag global. En la siguiente iteración del event loop, el servidor cierra los sockets pasivos (deja de aceptar nuevas conexiones) pero sigue atendiendo las conexiones existentes hasta que todas terminen. Una segunda señal fuerza el cierre inmediato.

3. Pruebas de estrés

Se desarrolló un script de estrés (`tests/stress_socks.py`) que levanta un servidor echo local y abre conexiones SOCKS autenticadas de forma concurrente a través del proxy, mezclando sesiones cortas (ráfaga de datos y cierre) con sesiones de larga duración (ping periódico). Esto permite evaluar el comportamiento del servidor bajo carga sostenida.

Se ejecutó la prueba con 500 conexiones concurrentes (450 cortas + 50 de larga duración) durante 60 segundos:

```
./bin/server -u alice:secret -u bob:pass &  
python3 tests/stress_socks.py --connections 500 --long-connections 50 \  
--duration 60 --steady-users --user-change-rate 10
```



```

[tiziano@usb ~/repos/itba/SOCKSv5-proxy]$ python3 stress_socks.py --connections 500 --long-connections 50 --duration 60 --steady-users --user-change-rate 10
local echo target listening on 127.0.0.1:45259
 5.0s attempts=500 connected=50 failed=0 active_long=50 active_short=0 closed=0 sent=2157 rcv=2157
10.0s attempts=500 connected=100 failed=0 active_long=50 active_short=50 closed=0 sent=398642 rcv=398642
15.0s attempts=500 connected=149 failed=0 active_long=50 active_short=99 closed=0 sent=820123 rcv=820123
20.0s attempts=500 connected=199 failed=0 active_long=50 active_short=149 closed=0 sent=1179346 rcv=1179346
25.0s attempts=500 connected=248 failed=0 active_long=50 active_short=198 closed=0 sent=1621003 rcv=1621003
30.0s attempts=500 connected=298 failed=0 active_long=50 active_short=248 closed=0 sent=2044905 rcv=2044905
35.0s attempts=500 connected=347 failed=0 active_long=50 active_short=297 closed=0 sent=2453465 rcv=2453465
40.0s attempts=500 connected=397 failed=0 active_long=50 active_short=347 closed=0 sent=2837040 rcv=2837040
45.0s attempts=500 connected=446 failed=0 active_long=50 active_short=396 closed=0 sent=3246041 rcv=3246041
50.0s attempts=500 connected=496 failed=0 active_long=50 active_short=446 closed=0 sent=3654200 rcv=3654200
55.0s attempts=539 connected=500 failed=0 active_long=50 active_short=405 closed=45 sent=3692769 rcv=3692769
60.0s attempts=589 connected=500 failed=0 active_long=50 active_short=355 closed=95 sent=3694039 rcv=3694039
final: attempts=589 connected=589 failed=0 closed=589 sent=4437609 rcv=4437609

```

Figura 1: Salida del script de estrés mostrando conexiones activas y bytes transferidos

```

[tiziano@usb ~/repos/itba/SOCKSv5-proxy]$ ./bin/client -u alice:secret -m
Historic connections: 1247
Current connections: 233
Bytes transferred: 17982540

```

Figura 2: Métricas del servidor durante la prueba de estrés (conexiones concurrentes y bytes)

- **Máxima cantidad de conexiones simultáneas:** con `pselect`, el límite teórico es $\lfloor \text{FD_SETSIZE}/2 \rfloor \approx 500$ conexiones SOCKS (cada una consume 2 fds: cliente + origen). En la prueba se alcanzaron 500 conexiones activas (50 de larga duración + 450 cortas) con 0 errores.
- **Comportamiento bajo carga:** las 500 conexiones transfirieron un total de ~ 4.4 MB en 60 segundos sin pérdida de datos. El servidor no rechazó conexiones ni produjo errores de I/O.
- **Memoria:** cada conexión consume ~ 8.5 KB (struct socks5 con dos buffers de 4096 bytes, parsers, mutex y flags). Con 500 conexiones, el consumo total de estructuras de conexión es de ~ 4.2 MB.

Para superar la limitación de `FD_SETSIZE` sería necesario migrar a `epoll`, que no tiene techo de file descriptors.

4. Problemas encontrados

- La compilación con `-std=c11 -pedantic` oculta las extensiones POSIX necesarias (`pselect`, `sigaction`, etc.). Se resolvió definiendo `_POSIX_C_SOURCE=200809L` en los flags de compilación.
- El `connect` no bloqueante retorna `EINPROGRESS` y requiere verificar el resultado posterior con `getsockopt(SO_ERROR)`. El manejo incorrecto de este caso resultó en conexiones que no se establecían correctamente.

- La estructura de conexión compartida entre el hilo DNS y el hilo principal generó problemas de use-after-free cuando el cliente cerraba antes de que terminara la resolución. Se resolvió con reference counting atómico.
- La señal **SIGPIPE** debe ignorarse explícitamente; de lo contrario, una escritura a un socket cerrado por el peer termina el proceso de forma silenciosa.
- La sincronización con el hilo DNS requirió un mutex para proteger el acceso al resultado de `getaddrinfo`. En la práctica el mutex es parcialmente redundante (el selector solo accede al resultado después de recibir la notificación del hilo), pero se incluyó como medida defensiva para evitar data races en casos borde.
- La serialización del registro de accesos en el protocolo de monitoreo presentó un problema de framing: al truncar la lista por tamaño de payload, se podían cortar entries a la mitad. Se resolvió con una doble pasada — primero se calcula cuántas entries completas caben en el buffer (recorriendo de la más nueva a la más antigua), y luego se serializan en orden cronológico (oldest-first) desde la posición de inicio calculada.

5. Limitaciones

- Máximo de 10 usuarios configurados simultáneamente.
- El registro de accesos almacena las últimas 1024 entradas (buffer circular).
- Las métricas y el registro de accesos son volátiles (se pierden al reiniciar).
- Se resuelve una dirección DNS a la vez por conexión.
- El uso de `pselect` limita la cantidad de file descriptors monitoreables a `FD_SETSIZE` (típicamente 1024). Con 500 conexiones SOCKS, cada una usando 2 fds (cliente + origen), se roza el límite. Para escalar más allá de esto sería necesario migrar a `epoll`.

6. Posibles extensiones

- Detección de credenciales POP3 en tránsito (segunda entrega).
- Persistencia de usuarios y métricas en disco.
- Pool de hilos para resolución DNS concurrente.
- Rate limiting por usuario.
- Lista de destinos bloqueados.

7. Conclusiones

El proyecto permitió aplicar en la práctica los conceptos de programación con sockets no bloqueantes, multiplexación con `pselect`, y diseño de protocolos binarios. La separación entre la capa de I/O y la lógica de protocolo resultó fundamental para mantener el código manejable a medida que creció la complejidad del relay y el apagado ordenado.

La infraestructura provista por la cátedra (selector, buffer, stm, parser) resultó adecuada para construir un servidor concurrente sin necesidad de dependencias externas.

8. Ejemplos de prueba

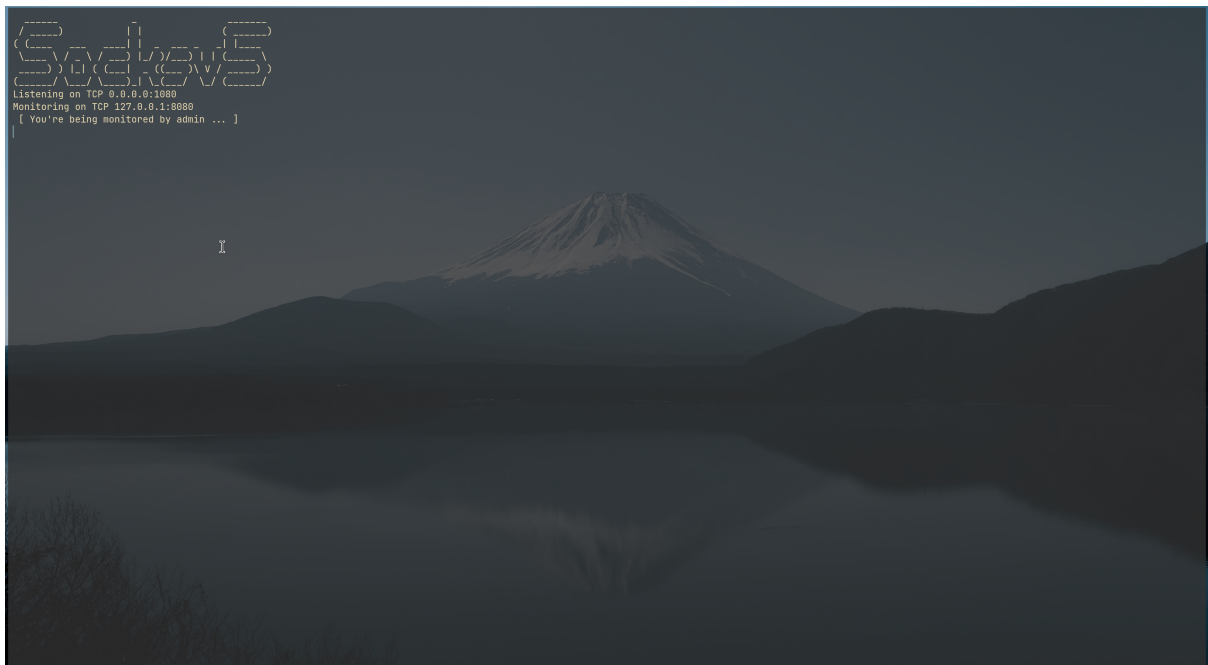


Figura 3: Servidor iniciado escuchando en ambos puertos

```
[tiziano@usb ~]$ curl -v --proxy socks5h://admin:pass@127.0.0.1:1080 http://google.com
* Trying 127.0.0.1:1080...
* Opened SOCKS connection from 127.0.0.1 port 50196 to google.com port 80 (via 127.0.0.1 port 1080)
* Established connection to 127.0.0.1 (127.0.0.1 port 1080) from 127.0.0.1 port 50196
* using HTTP/1.1
> GET / HTTP/1.1
> Host: google.com
> User-Agent: curl/8.20.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 301 Moved Permanently
< Location: http://www.google.com/
< Content-Type: text/html; charset=UTF-8
< Content-Security-Policy-Report-Only: object-src 'none';base-uri 'self';script-src 'nonce-446r8HgImKj_Y2A4Cy3tw' 'strict-dynamic' 'report-sample' 'unsafe-eval' 'unsafe-inline' https: http:;report-uri https://csp.withgoogle.com/csp/gws/other-hp
< Date: Tue, 07 Jul 2026 01:18:16 GMT
< Expires: Thu, 06 Aug 2026 01:18:16 GMT
< Cache-Control: public, max-age=2592000
< Server: gws
< Content-Length: 219
< X-XSS-Protection: 0
< X-Frame-Options: SAMEORIGIN
<
<HTML><HEAD><meta http-equiv="content-type" content="text/html; charset=utf-8">
<TITLE>301 Moved</TITLE></HEAD><BODY>
<H1>301 Moved</H1>
The document has moved
<A HREF="http://www.google.com/">here</A>.
</BODY></HTML>
* Connection #0 to host google.com:80 left intact
```

Figura 4: Conexión exitosa a través del proxy con curl

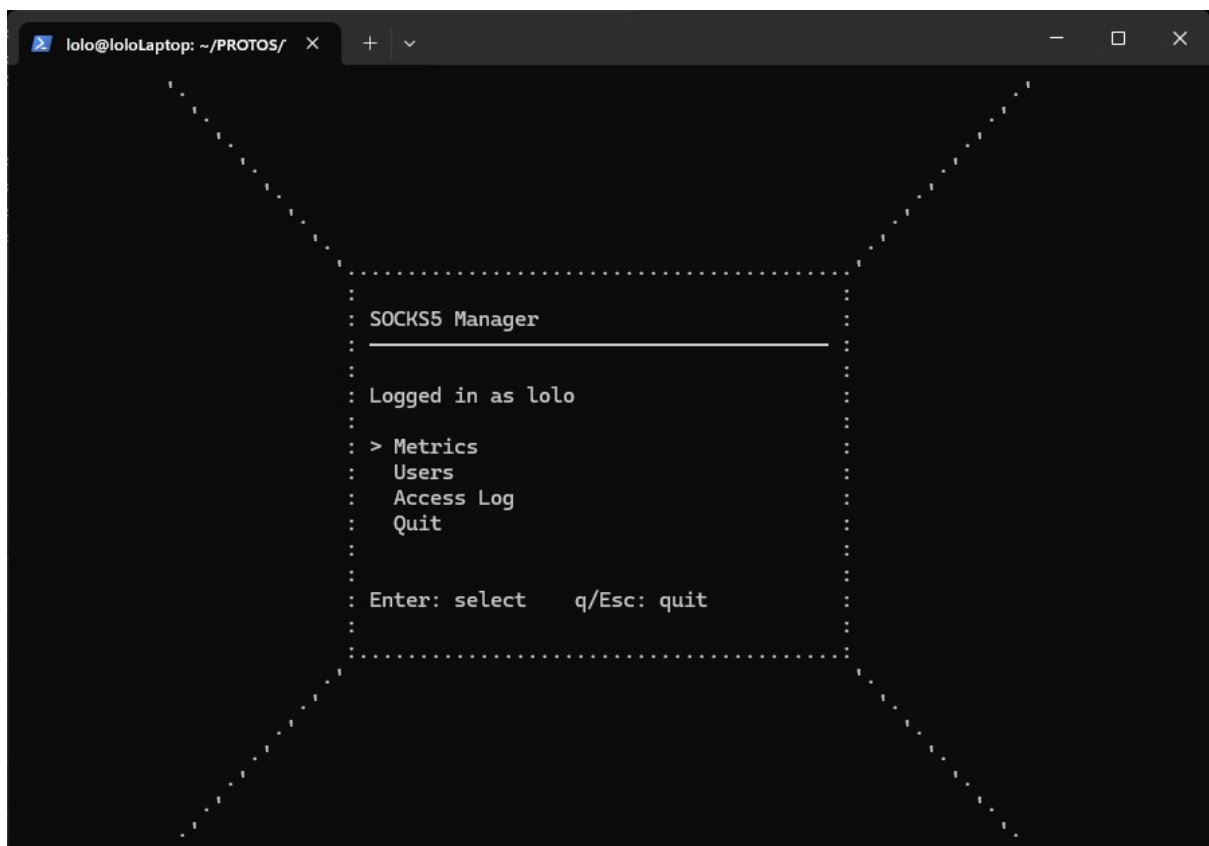


Figura 5: Cliente de monitoreo en modo interactivo

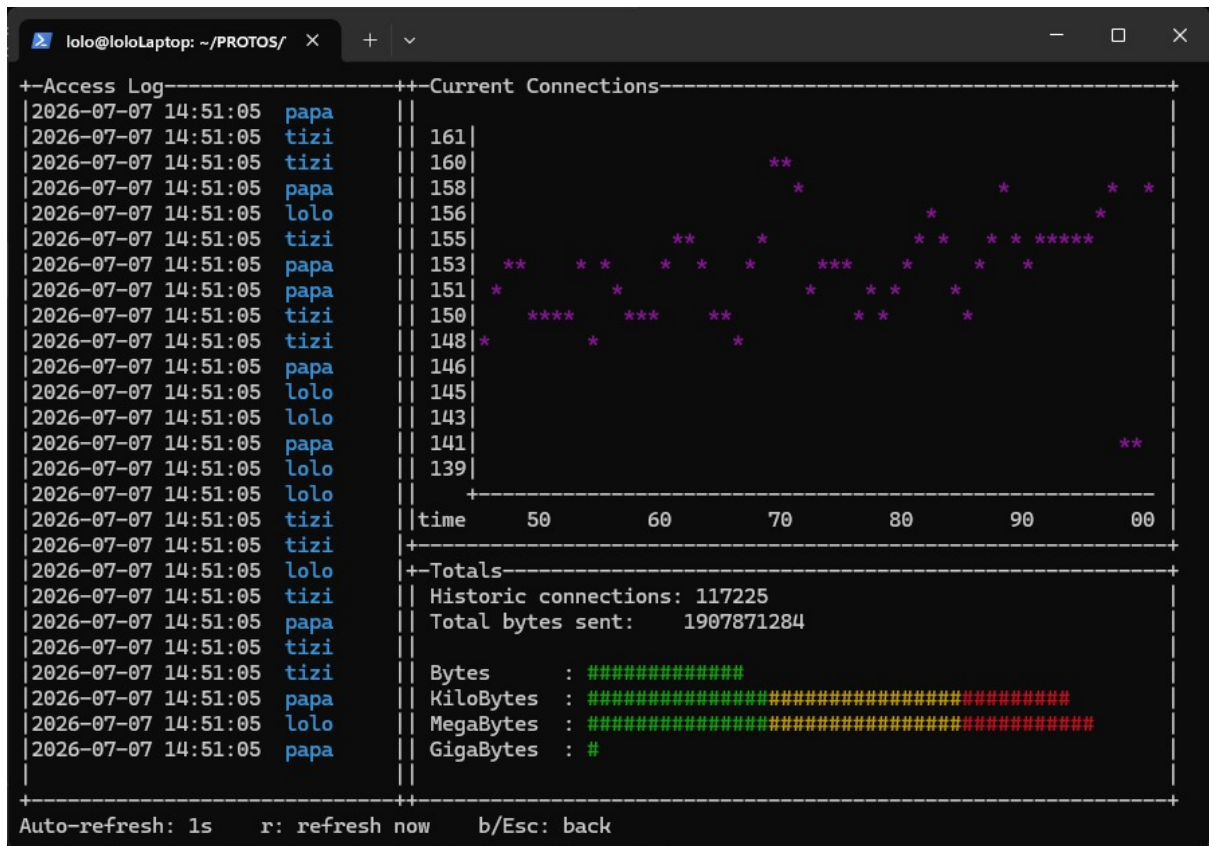


Figura 6: Vista de métricas desde el cliente

```
lolo@loloLaptop: ~/PROTOS/ × + v
+-Users-----+
> papa
tizi
lolo

+-----+
|          | Name   : papa
|          | Password: (unknown)
|          | Last connect:
|          | 2026-07-07 14:51:14
|          | 127.0.0.1:33607
|          |
+-----+
[x] Terminate user

[a] Add a user

Up/Down: scroll users  a: add user  x: terminate user  r: refresh  b/Esc: back
```

Figura 7: Vista de gestión de usuarios

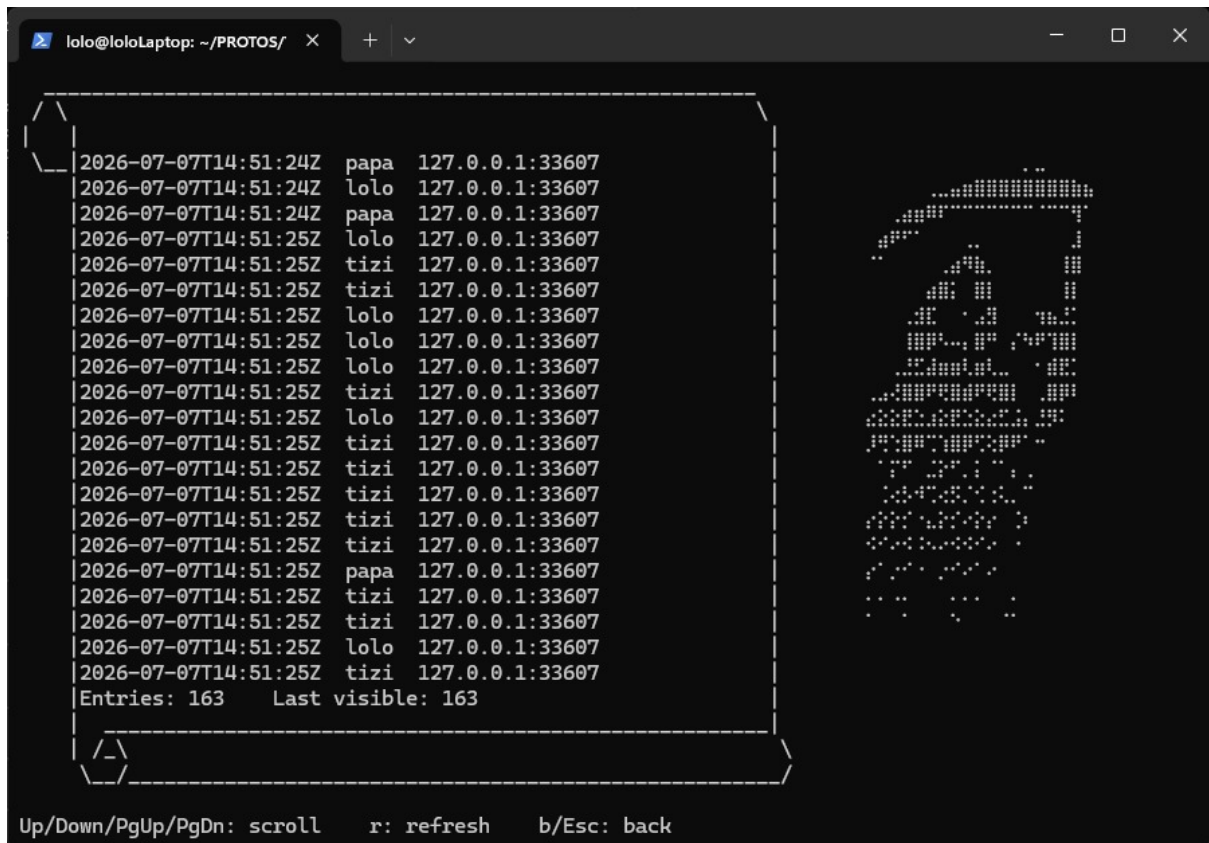


Figura 8: Vista del registro de accesos

9. Guía de instalación

Dependencias: gcc con soporte C11, pthreads (incluido en glibc), libncursesw (para el cliente) y libcheck (para tests).

```
make          # compila bin/server y bin/client
make test     # compila y ejecuta los tests
make clean    # limpia objetos y binarios
```

10. Configuración

El servidor se configura por línea de comandos:

Opción	Default	Descripción
-l addr	0.0.0.0	Dirección de escucha SOCKS
-p port	1080	Puerto SOCKS
-L addr	127.0.0.1	Dirección de escucha monitoreo
-P port	8080	Puerto monitoreo
-u user:pass	—	Agregar usuario (hasta 10)
-U file	—	Cargar usuarios desde archivo

11. Ejemplos de configuración y monitoreo

```
# Servidor con dos usuarios en puerto no estándar
./bin/server -p 9090 -u alice:secret -u bob:pass123
```

```
# Consultar métricas
./bin/client -u alice:secret -P 8080 -m
```

```
# Agregar usuario en tiempo de ejecución
./bin/client -u alice:secret -a charlie:nueva
```

```
# Consultar registro de accesos
./bin/client -u alice:secret --access-log
```